SOFTWARE ARCHITECTURE — CHAPTER 9

Deployability, Portability, and Containers

Shipping the same artifact from laptop to cluster — safely and often

Based on Pautasso — Visual Lecture Notes

Chapter PDF: `09_Deployability_Portability_and_Containers.pdf`

RECAP — LECTURE 8

Compatibility & Coupling

Chapter 8: coupling strength, compatibility, and safe API evolution

Examples: `example1_compatibility_and_api_evolution.py`, `example2_coupling_and_dependencies.py`

→ Today: **deployability** (how we ship), **portability** (where it runs), and **containers** as the packaging unit.

Recap: coupling

Coupling = how much one part depends on another's details (schemas, call order, deployment timing).

- Strong coupling → small provider changes **ripple** to many consumers
- Architecture views should show dependency **direction** and hotspots (shared DB, DTOs)

Chapter 8 asked you to **name** coupling types when analyzing the Task Board API.

Recap: compatibility

Syntactic

Wire shape parses: fields, types, required vs optional.

Semantic

Meaning: validation, idempotency, error `code` semantics—can break without JSON renames.

Backward compatibility is the usual goal for public APIs and long-lived partners.

Recap: semver & coexistence

- **MAJOR / MINOR / PATCH** — align policy with what clients actually tolerate
- **v1 + v2** — paths, headers, gateways; **sunset** windows and deprecation notices
- A written **policy** turns ad hoc API decisions into team rules

Immutable **container images** pin what ran in prod—but config and secrets still evolve.

Recap → Chapter 9

- Lecture 8: **what** we expose and **whether** changes are safe for consumers
- Lecture 9: **how often** we ship, **where** binaries run, and **what the platform** (e.g. Kubernetes) guarantees
- Running example this week: **CityBite** — food delivery moving from VMs to containers

Learning objectives

- Define **deployability** and list factors that speed or block safe releases
- Explain **portability** traps: paths, config, secrets, writable filesystems
- Describe **containers** vs VMs; images, layers, immutability at a high level
- Apply **twelve-factor style** practices: `PORT`, env config, stdout logs, disposability
- Contrast **liveness** vs **readiness** and relate to rolling updates
- Separate **stateless** app tiers from **data** (DB, uploads) in cluster designs

From design time to production

After **design-time** qualities (modularity, interfaces, composition, compatibility), we **deploy** the system.

- This lecture: **deployability** (how easy/safe to ship) and **portability** (run everywhere)
- **Virtualization & containers** — standard packaging and isolation for cloud runtimes
- **Next** in the course arc (per chapter): **scalability, availability, flexibility** — deeper runtime qualities

Pautasso: *"This is the lecture in which we focus on how easy it is to deploy… and virtualization and containers… with an eye towards portability."*

Chapter roadmap

- **Deployability metrics** — latency, throughput, downtime, cost of releases
- **To change or not to change** — opportunity vs risk; reversibility; Big Bang vs continuous
- **CI, continuous delivery, continuous deployment** — the software **production pipeline**
- **Platform independence** — VMs, hypervisors, **containers**, images, migration

The age of continuity

Release cadence shifted from **years** (e.g. major desktop OS) to **continuous** updates in the cloud.

- Users may **no longer perceive a "version"** — each visit can hit a slightly newer system
- Industry example (lecture): providers moved from ~**10 releases / 10s** to **~1 / second** scale (circa 2011→2014 narrative)
- **Challenge:** high frequency **without** sacrificing correctness and other qualities

CityBite's **dinner spikes** make safe, frequent deploys a business requirement—not only an engineering preference.

Deployability metrics — latency & throughput

Deployability can be **measured** like other quality attributes.

- **Latency (forward):** dev commits → user **sees** feature or **bugfix** in production
- **Latency (backward):** user feedback → triage → fix → user **accepts** the improvement
- **Throughput:** releases per **hour / day / month** — from “once every few years” to **many per day**

Deployability metrics — time, cost, pipeline

- **Downtime:** how long are users blocked during upgrade? (maintenance page vs seamless)
- **Deadlines:** marketing launch (e.g. holiday) — slip = revenue risk
- **Human cost:** one-click vs multi-team “release weekend” coordination
- **Infrastructure cost:** laptop build vs **cluster** of VMs for parallel compile + hours of tests

Huge mono-repos: “wait until tomorrow’s nightly” **limits** developer feedback—deployability of the **architecture** and build.

Traditional vs continuous delivery

	Traditional	Continuous
Latency	High	Small
Throughput	~1 / years	Many / day
Downtime	Often noticeable	Target **zero** perceived
Deadlines	“Death march” pressure	No single D-Day window
People	Release engineering craft	**Automation** (toil → code)

Lecture: releases matured from **craft** to **industrial** workflow—when tooling and culture allow.

Every release: opportunity *and* risk

Opportunity

Features, fixes, performance, revenue, retention, competitive catch-up.

Risk

New bugs/failure modes, capacity cost, support & training load, **bad deploy**, **unrecoverable** state (data loss).

Michael T. Nygard — *Release It!*; operators may fear change for good reason—**design** for rollback.

No change ≠ no risk

- Only **unused** software never needs to change
- Goal: **minimize risk *of* change** — not “minimize change” forever
- **Small** deltas, **frequent** practice, **reversible** steps (like DB **transactions**: commit / rollback)

“Weekends have two days: Saturday deploy, Sunday **revert**” — plan **time buffer** and recovery strategy.

Big Bang vs continuous flow

- **Classic waterfall handoff:** dev “done” → ops installs; **first** real test is often production load
- **“If it hurts, do it more often”** — reduce batch size and rehearse the transition
- **Continuous:** build & operate **in parallel**; ops feedback loops to dev; red deploys are **expected** and fixed fast

Speed vs quality — automation reframes the tradeoff

- Old assumption: **fast** shipping ⇒ **low** quality; high quality ⇒ **slow**
- Continuous delivery **cannot** sacrifice quality every push—would live in a broken state
- **Automated QA** + repeatable packaging ⇒ **high throughput** and **high** confidence together

The **pipeline** that builds software is **software** — version, review, and test it too.

Software production pipeline

Input: source repo → **output:** immutable **artifact** (image) ready for users.

- Stages: **build** (compile + **unit** tests + static analysis) → **integrate** → deeper tests (**perf**, **acceptance**)
- **Release decision** — human or automated gate → **deploy** → **smoke** / readiness in prod
- Word that repeats in the chapter: **test** — encode expectations so machines can enforce them every run

CI · Continuous Delivery · Continuous Deployment

Term	Extends to...
Continuous Integration	Frequent merge; automated **build + unit/integration** to a runnable integrated product
Continuous Delivery	Artifact **fully tested** and packaged; **manual** approval before production deploy
Continuous Deployment	**Automatic** deploy when checks pass — needs exceptional test coverage; **mobile** stores add delay

High quality at high speed — practices

- Everyone **sees** the pipeline; **don't push** on broken build; **watch** until green; don't go home on red
- **Small** commits; **fast** feedback on push; **nightly** heavier suites when needed
- **Reproducible** builds — **snapshot dependencies**; **immutable** released images (no hot-edits in prod)
- **No** "comment out failing tests"; **time-box** fix then **revert**; **forbid** un-tracked prod edits—fix at **source**

Jez Humble & David Farley — *Continuous Delivery*.

Types of testing

Type	Purpose
Unit	Components in isolation (mocks)
Integration	System with real dependencies wired
User / E2E	Use cases via UI — often partly manual
Capacity / perf	Resources vs targets; cost of a workload
Acceptance	Meets agreed criteria — **ship / no-ship**
Smoke	Quick prod/staging sanity after deploy

Testing in production always exists; pre-release tests decide **how often** users are first to find bugs.

Release strategies

- **Big Bang / plunge** — everyone switches; risky rollback
- **Blue / green** — instant traffic flip; keep old stack for **rollback** (2× capacity cost)
- **Shadow / dark launch** — new version consumes **real** input; output hidden; compare behavior
- **Pilot / canary / gradual / A/B** — ramp trusted or random users; measure metrics before full cutover

Gradual strategies require **multiple active versions** — limits how many trains you can run in parallel.

Virtualization

- **Hypervisor** multiplexes **guest OS + apps** over **physical hardware** — elasticity, migration, isolation
- **Hosted** hypervisor (e.g. laptop: host OS + VirtualBox) vs **native** hypervisor on bare metal (datacenter)
- **Live migration** — move VMs between hosts for maintenance or failure—**impossible** with pets-on-metal alone

VM vs container

	VM	Container
Footprint	Often GB	Often MB + app
Boot	Minutes typical	Sub-second to seconds
Guest OS	Per VM	Shared host kernel
Isolation	Hardware-assisted	Namespaces / cgroups (OS sandboxing)
Persistence	Often stateful disk image	Treat workload as **stateless**; data outside

Containers in VMs — common cloud pattern: rent VM, run many containers for density and parity.

Images, snapshots, migration

- **Image** — frozen filesystem + metadata to boot a known state (VM or container)
- **Snapshots / checkpoints** — capture RAM + disk for resume or disaster recovery
- **Repository** — versioned artifacts; **digest** = what actually ran in the incident postmortem

Virtual hardware as software

- CPU/RAM/disk/network **allocation** lives in **versioned config** (Terraform, Helm, K8s YAML)
- **Never mix** production and test environment definitions—same discipline as application code
- **Inverted hypervisor** (advanced): one logical machine spread over **many** hosts for huge workloads

This closes the arc from design → **packaged, configured** runtime—next: scale, availability, flexibility.

Deployability

Deployability = how easily and safely you can move a change from commit to production—and roll back if needed.

- **Automation** — CI builds tests and artifacts; CD promotes known-good images
- **Frequency & blast radius** — smaller diffs, feature flags, canary or rolling updates
- **CityBite pain** — SSH to long-lived VMs, dinner-rush deploys, minutes of partial downtime during restarts

Architecture choices (state, sessions, disk writes) directly constrain how often you can deploy.

Portability

Portability = the same application artifact runs in dev, staging, and prod without one-off hacks per host.

- **Configuration** — environment variables, mounted config maps—not secrets baked into images
- **Filesystem** — avoid hardcoded OS paths (`C:\...`) and “always writable cwd” assumptions
- **Dependencies** — pinned libraries; reproducible builds from lockfiles / image layers

Containers help portability but do not remove bad path habits—see Example 2 (`DATA_DIR`).

Containers (high level)

- **Image** — immutable template: OS userland slice, runtime, app files, metadata
- **Container** — running instance from an image: isolated process tree, namespaces, cgroup limits
- **Orchestrator** (e.g. Kubernetes) — schedules replicas, replaces failed nodes, exposes Services/Ingress

Treat the image digest as “what actually ran” when debugging production incidents.

Images, layers, immutability

- **Layers** — cache dependency installs; smaller layers = faster pull on new nodes
- **Immutable servers** — replace containers instead of patching live filesystems ad hoc
- **Secrets** — inject at runtime (K8s Secret, cloud secret manager); never `docker commit` credentials into an image

Multi-stage builds keep compile toolchains out of the final runtime image (smaller attack surface).

Twelve-factor style (selected)

Config & port

`DATABASE_URL`, `LOG_LEVEL`, `PORT` from the environment—what Fargate/EKS inject.

Logs & processes

Stream logs to **stdout/stderr**; one main process; handle **SIGTERM** for graceful drain.

Example 1 contrasts laptop anti-patterns with what CityBite SRE expects in the cloud.

CI → registry → cluster

- Push on main → tests → build image tagged with `git sha` → promote to `:v1.5.0`
- Cluster pulls by digest or immutable tag; roll out **Deployment** with rolling strategy
- **Rollback** — keep previous ReplicaSet; `kubectl rollout undo` or redeploy last known-good tag

Your assignment sequence diagram should show this path for CityBite.

Kubernetes in one slide

Object	Role
Pod	One or more co-scheduled containers sharing network/storage volumes
Deployment	Desired replicas + rolling update policy for stateless apps
Service	Stable cluster IP / DNS to reach healthy pods
Ingress	HTTP routing, TLS termination at the edge
PVC	Claim on durable disk for uploads or local caches (vs object storage)

Liveness vs readiness

Liveness

"Is the process wedged?" — failure → **restart** the container.

Readiness

"Can this pod take traffic?" — DB down → **remove** from Service until recovered.

Wrong probes cause restart storms or send traffic to pods that are still starting—design thresholds carefully.

State: CityBite menu photos

- **App pods** should be disposable—scale out/in without copying disk by hand
- **Menu JPEGs** — today on VM disk; in K8s use **PVC +** `DATA_DIR` or **S3** (compare in assignment)
- **Postgres** — typically managed **outside** the cluster; pods get `DATABASE_URL`

Example 2 shows why `./uploads` fails on read-only or ephemeral container roots.

Pets vs cattle & pitfalls

- **Pets** — snowflake servers with manual fixes; hard to reproduce
- **Cattle** — replaceable nodes and pods; config and data live **outside** the disposable unit
- **Pitfalls** — sticky sessions in memory, local file sessions, writing outside mounted volumes

Moving CityBite off VMs is as much **culture** (IaC, runbooks) as it is Docker syntax.

Security & ops hygiene

- Run as **non-root** in the container where possible; read-only root FS + explicit writable paths
- Scan images for CVEs; patch base images on a schedule
- NetworkPolicy / service mesh (optional) to limit lateral movement between namespaces

Align with your organization's platform team—Chapter 9 connects architecture to **shared responsibility**.

Course examples (Python) — CityBite

- `example1_deployability_citybite_api.py` — order API: anti-pattern (`C:\...` paths, fixed port) vs `PORT` , `DATABASE_URL` , JSON logs on stdout for ECS/K8s
- `example2_portability_menu_uploads.py` — restaurant **menu uploads**: `./uploads` vs `DATA_DIR` for volumes / temp in containers

Run both after class; deep dive in `09_Deployability_Portability_and_Containers.pdf` .

ASSIGNMENT

CityBite on Kubernetes

VM monolith → Kubernetes; containers, probes, portability

Full spec: `ASSIGNMENT.md` — this section orients you; it does **not** solve the assignment.

Assignment: scenario

CityBite — regional food delivery: customer + rider + restaurant apps; **dinner spikes**; **menu JPEGs** on VM disk today.

- **Baseline:** monolith on long-lived VMs, SSH deploys, Postgres managed, uploads under `/var/citybite/uploads`
- **Goal:** containerized services on **Kubernetes** (pick AWS/GCP/Azure), CI-built images, env-based config
- You deliver **markdown + diagrams** (Dockerfile sketch in fenced code only)—no running cluster required

Reuse env var names from the lecture examples where they apply (`PORT` , `DATABASE_URL` , `DATA_DIR` , …).

What you produce (big picture)

Area	You decide & justify
Deployability	Risks of today's VM model + mitigations in K8s
Architecture	Before/after diagram (VM vs cluster)
Containers	Image/layers, process model, Dockerfile sketch, probes
Portability	Uploads (PVC vs S3), secrets, DB connection, local dev parity
Delivery	Sequence CI → registry → rollout (≥6 participants)

Part 1 — Deployability assessment

Task 1.1

- ≥5 deployability risks (host drift, manual SSH, disk coupling, rollback time, …)
- Per risk: one **Kubernetes/container**-shaped mitigation (specific, not “use K8s” alone)
- One thing that gets **harder** after migration and how you address it

Deliverable: `part1_deployability_assessment.md` — **30 pts**

Part 1 — Before / after diagram

Task 1.2

- **draw.io:** left = VM monolith + DB + disk uploads; right = cluster (Deployments, Service, Ingress, PVC or S3, managed DB)
- Label **two** request/data paths; include **legend**

Deliverables: `part1_architecture_before_after.drawio` + `.png` — **20 pts**

Part 2 — Container spec

Task 2.1

- Order API image (required) + optional worker: base image, build steps, non-root if applicable
- Runtime contract: env vars (`PORT` , `DATABASE_URL` , ...), logging to stdout, single main process
- Short **Dockerfile** sketch (10–25 lines) for API only—in markdown

Deliverable: `part2_container_spec.md` — **25 pts**

Part 2 — Health & rollout

Task 2.2

- **Liveness** vs **readiness** definitions for Order API (paths/thresholds)
- Rolling update `v1.4.0` → `v1.5.0`; what if new pods never go Ready?
- Short **incident** paragraph: detect bad deploy + rollback story

Deliverable: `part2_health_and_rollout.md` — **15 pts**

Part 3 — Portability & state

Task 3.1

- **Menu uploads:** PVC + `DATA_DIR` **vs** object storage — two pros/cons each for CityBite
- **Secrets** and **DATABASE_URL** injection; DB outside cluster (or justify alternative)
- **Dev/prod parity:** e.g. docker compose with volume for uploads

Deliverable: `part3_portability_and_state.md` — **20 pts**

Part 3 — Delivery sequence

Task 3.2

- Sequence diagram: **≥6** actors (dev, git, CI, registry, control plane, pod)
- Include image tag flow (`git sha` → release tag) + **one failure branch**

Deliverables: `part3_delivery_sequence.drawio` + `.png` — **10 pts**

Submission & grading

- **Pull Request** to `lecture-9/submissions/YOUR_NAME/` — `../lecture-3/SUBMISSION_GUIDE.md`
- Folder `README.md` : your name, cloud choice, assumptions
- **120 points** — rubric in `ASSIGNMENT.md` ; every diagram: `.drawio` + `.png`

Before you start

Hints — not answers

1. Run `example1_deployability_citybite_api.py` and `example2_portability_menu_uploads.py`
2. Pick **one** cloud + Kubernetes distribution for the whole submission
3. Ask: “Where does **dinner rush** traffic hit first—Ingress, HPA, DB connection pool?”
4. Keep **CityBite** naming consistent across every file you submit

Takeaways

- **Deployability** is automation + architecture (state, sessions, disk)
- **Containers** standardize packaging; **config & secrets** still need discipline
- **Portability** fails on paths, writable roots, and implicit host state
- **Probes & rollouts** are part of the contract between dev and platform teams
- **Data** (Postgres, menu files) needs an explicit home outside disposable pods

Questions?

Chapter 9 — Deployability, Portability, and Containers

Deep dive: `09_Deployability_Portability_and_Containers.pdf`

Follow your course schedule for the next chapter and assignment deadlines.